MANUAL DE DUDAS CRÍTICAS RESUELTAS

Guía de Correcciones Mecánicas para el Examen de Laboratorio - IISSI1

Este documento recopila las dudas, confusiones frecuentes y tropiezos de sintaxis que hemos ido identificando y resolviendo a lo largo de tus sesiones de práctica. Está diseñado para ayudarte a recordar de forma visual e inmediata qué NO hacer y cómo resolverlo mecánicamente en HeidiSQL.

1. ¿Por qué poner "AS alias" en un SELECT no guarda el dato en mi variable?

Cuando haces un `SELECT columna AS mi_variable`, el uso de `AS` solo sirve para cambiar el nombre que se muestra en la cabecera de la pantalla de HeidiSQL. No guarda nada en el disco ni en la memoria del procedimiento.

```
-- ERROR CONCEPTUAL DE EXAMEN:
SELECT lineaspedido.productoId AS listaIdProductos FROM lineaspedido;
```

La Regla Mecánica: Para almacenar un dato dentro de una variable local previamente declarada, se debe usar obligatoriamente la palabra clave **`INTO`**.

```
SELECT precio INTO v_precio FROM Productos WHERE id = p_productoId;
```

2. ¿Puedo meter una lista de varios IDs en una variable normal de tipo INT?

No, nunca. Una variable de tipo `INT` es una celda de memoria que solo puede contener un único número entero a la vez. Si el `SELECT ... INTO` devuelve más de una fila (por ejemplo, los IDs de los 3 productos de un pedido), el procedimiento colapsará inmediatamente en ejecución lanzando un error.

La Regla Mecánica: En lugar de intentar extraer una lista de IDs con variables para hacer un UPDATE, se debe juntar la tabla destino con la tabla de origen usando un **`JOIN` directo dentro del propio `UPDATE`**.

```
UPDATE Productos p
JOIN LineasPedido l ON p.id = l.productoId
SET p.stock = p.stock + l.unidades
WHERE l.pedidoId = numPedido;
```

3. El peligro de vaciar el almacén: ¿Por qué "SET stock = 0" está mal al anular o devolver?

Al anular o cancelar un pedido, los artículos comprados vuelven físicamente a las estanterías de la tienda. Si pones `SET stock = 0`, estás destruyendo todo el inventario restante del producto. Lo correcto es realizar una operación aritmética sumando las unidades devueltas.

```
-- ERROR DE LÓGICA DE NEGOCIO (0 puntos en el apartado):
UPDATE productos SET stock = 0 WHERE id IN (...);
```

La Regla Mecánica: Modifica el stock de forma matemática basándote en lo que ya había en la base de datos:

```
SET p.stock = p.stock + l.unidades
```

4. ¿Para qué sirven exactamente "NEW" y "OLD" en los Triggers?

Son pseudotablas que MariaDB te regala para inspeccionar los datos que están mutando en la tabla. Su disponibilidad cambia según el evento:

- **INSERT:** Solo existe **`NEW`** (los datos del futuro que van a entrar). `OLD` no existe porque no había nada antes.
- **DELETE:** Solo existe **`OLD`** (los datos del pasado que se van a destruir). `NEW` no existe porque la fila desaparece.
- **UPDATE:** Conviven **ambos**. `OLD` tiene la fila tal y como estaba en el disco duro (pasado) y `NEW` tiene las modificaciones que el usuario intenta guardar (futuro).

Ejemplo en UPDATE: Para comprobar si el usuario intentó cambiar un campo protegido, los comparas en el `IF`:

```
IF (NEW.clienteId <> OLD.clienteId) THEN ...
```

5. El Error de Mutación en Triggers: No hagas SELECT sobre la tabla que disparó el evento

Si estás dentro de un trigger `BEFORE INSERT ON LineasPedido`, la palabra `NEW.unidades` ya contiene directamente el valor que introdujo el usuario. Si intentas hacer un `SELECT unidades FROM LineasPedido` dentro de ese cuerpo, el motor bloqueará la base de datos porque la tabla está cambiando en ese mismo instante.

```
-- ERROR DE COMPILACIÓN (Bloqueo de tabla mutante):
SELECT new.unidades INTO v_unidades FROM lineaspedido;
```

La Regla Mecánica: No declares variables ni hagas consultas de extracción para los datos de la fila actual. Léelos directamente usando `NEW.nombre_columna` dentro de las condiciones.

```
IF NOT (NEW.unidades > 0 AND NEW.unidades <= 100) THEN ...
```

6. ¿Cómo evito colocar comas letales o desordenar las declaraciones (DECLARE)?

En los procedimientos almacenados (Procedures), el orden de la zona de configuración inicial (justo bajo el primer `BEGIN`) es estricto. Cada sentencia de declaración debe ser completamente independiente y terminar con su propio punto y coma `;`. Las comas entre distintos `DECLARE` rompen el compilador.

```
-- ERROR DE SINTAXIS (Mezclar declaraciones con comas):
DECLARE EXIT HANDLER FOR SQLEXCEPTION,
DECLARE v_precio DECIMAL(10,2);
```

La Regla Mecánica: Sigue siempre este orden estructural aislado:

```
BEGIN
    -- 1º: Todas las variables de trabajo independientes
    DECLARE v_precio DECIMAL(10,2);

    -- 2º: El bloque Airbag del manejador de excepciones
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error';
    END;
```

7. La Trampa del Error 1093 en MariaDB (DELETE con Subconsultas)

MariaDB prohíbe terminantemente que hagas un `DELETE FROM tabla_A` si dentro del `WHERE ... IN (SELECT ... FROM tabla_A)` estás volviendo a leer de esa misma `tabla_A`.

```
-- ERROR 1093 (Can't specify target table for update in FROM clause):
DELETE FROM lineaspedido WHERE id IN (
    SELECT lineaspedido.id FROM lineaspedido JOIN pedidos...
);
```

La Regla Mecánica: En lugar de filtrar por el ID de la propia tabla que vas a borrar, filtra utilizando el campo de la clave ajena (FK) apuntando a la tabla vecina. Así el `SELECT` interno no toca la tabla que se está destruyendo.

```
-- Solución perfecta para saltarse la trampa:
DELETE FROM LineasPedido
WHERE pedidoId IN (SELECT id FROM Pedidos WHERE clienteId = p_clienteId);
```

8. Funciones de Agregación: El peligro de devolver múltiples filas flotantes

Si realizas una operación matemática fila por fila como `SELECT unidades * precio FROM LineasPedido WHERE pedidoId = 5;`, la pantalla mostrará tantas filas independientes como artículos tenga el pedido. Si el enunciado te pide calcular el "Importe Total", debes agruparlas en un único valor acumulado.

La Regla Mecánica: Envuelve siempre la operación dentro de la función de agregación **`SUM(...)`** para comprimir todas las filas en un único número final y vuelca el resultado en tu variable local mediante el `INTO`.

```
SELECT SUM(unidades * precio) INTO v_total
FROM LineasPedido
WHERE pedidoId = p_pedidoId;
```